

```
print(oracle)

grover = Grover(oracle)

result = grover.run(backend, shots=1024)

counts = result['measurement']

print('\n4SAT with Grovers Search')
print('-----\n')
print('4SAT: ', expression)
```

October 12, 2021

SOLVING K-SAT PROBLEMS WITH QISKIT

Introduction

Interested in learning how to program quantum computers? Then check out our Qiskit textbook **Introduction to Quantum Computing with Qiskit** (<https://quantumcomputinguk.org/shop/introduction-to-quantum-computing-with-qiskit-ebook-1>).

In this tutorial we will explore how to solve k-SAT problems using Grovers Search on IBM's Quantum computers in Qiskit.

What is the k-SAT problem?

The k-SAT problem is a Boolean satisfiable problem with k literals. Given a Boolean formula consisting of a number of clauses each with k literals the goal is to find what inputs will satisfy the problem.

For example take the following formula:

$$F = (\bar{A} \vee B \vee C) \wedge (A \vee \bar{B} \vee C) \wedge (\bar{A} \vee B \vee C) \wedge (\bar{A} \vee B \vee \bar{C})$$

This is a k-SAT problem consisting of 4 clauses and 3 literals in each clause. The clauses are the terms encased in brackets while the literals are the variables in each clause. The overline above a variable denotes a negation of that variable. The symbol $\wedge$ denotes the AND operation while $\vee$ denotes the OR operation. Because there is 3 literals in each clause this k-SAT problem is known as a 3-SAT problem.

As the number of variables and clauses increases in the expression the more harder it is for a classical computer to evaluate the expression.

However using Grover's search algorithm we can find the variable combinations needed that will satisfy the boolean expression. Grover's algorithm has a huge advantage over classical methods as the time complexity is only $O(\sqrt{n})$.

Implementation

In Qiskit solving a k-SAT problem with Grover's Search can be done with the following steps:

Step 1: Encode the k-sat problem in to DIMACS format

The first step is to express the k-SAT problem in to DIMACS CNF format. DIMACS CNF is a format used in Qiskit to define Boolean expressions.

For example consider the following 4-SAT problem:

$$F = (\bar{A} \vee B \vee C \vee D) \wedge (A \vee \bar{B} \vee C \vee D) \wedge (A \vee B \vee \bar{C} \vee D) \wedge (A \vee B \vee C \vee \bar{D})$$

In DIMACS CNF format this can be:

```
expression = '''
c 4-SAT
p cnf 4 5
-1 2 3 4 0
1 -2 3 4 0
1 2 -3 4 0
1 2 3 -4 0
1 2 -3 -4 0
'''
```

Where:

- c denotes a comment
- p denotes the start of the problem line
- cnf denotes the problem type (conjunctive normal form)
- 4 and 5 after cnf denote the number of variables and clauses
- 0 marks the end of each clause in the expression

Obviously given the number of clauses this can be solved rather quickly. Some good examples of harder k-SAT problems can be found here:

<https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html>
(<https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html>)

Step 2: Specify the oracle and pass to the Grover function

The first step is to specify the correct oracle. Since we are evaluating a k-SAT problem you will need to use the LogicalExpressionOracle function:

```
oracle = LogicalExpressionOracle(expression)
```

Then pass the oracle in to the Grover function:

```
grover = Grover(oracle)
```

Step 3 : Run the algorithm

Next we just need to run the algorithm using grover.run():

```
result = grover.run(backend, shots=1024)
```

Where backend is the quantum device that the algorithm is run on and shots is the number of times we run the algorithm.

Step 4: Obtain the results

Next we need to get the results using the following code:

```
counts = result['measurement']
```

How to run the program

Copy and paste the code below in to a python file

1. Enter your API token in the IBMQ.enable_account('Insert API token here') part
2. Save and run

Code

```
from qiskit import IBMQ
from qiskit.aqua.algorithms import Grover
from qiskit.aqua.components.oracles import LogicalExpressionOracle

IBMQ.enable_account('ENTER API KEY HERE')
provider = IBMQ.get_provider(hub='ibm-q')

backend = provider.get_backend('ibmq_qasm_simulator')

expression = '''
c 4-SAT
p cnf 4 5
-1 2 3 4 0
1 -2 3 4 0
1 2 -3 4 0
1 2 3 -4 0
1 2 -3 -4 0
'''

oracle = LogicalExpressionOracle(expression)

grover = Grover(oracle)

result = grover.run(backend, shots=1024)

counts = result['measurement']

print('\n4SAT with Grovers Search')
print('-----\n')
print('4SAT: ', expression)
print('\nResults ', counts)
print('\nPress any key to close')
input()
```

Output

```
4SAT with Grovers Search
-----
4SAT:
c 4-SAT
p cnf 4 5
-1 2 3 4 0
1 -2 3 4 0
1 2 -3 4 0
1 2 3 -4 0
1 2 -3 -4 0

Results: ('0000': 1, '0001': 305, '0010': 212, '0011': 4, '0010': 592, '0011': 9, '0110': 2, '0111': 6, '1000': 190, '1001': 5, '1010': 1, '1011': 3, '1100': 102, '1101': 5, '1110': 4, '1111': 4)

Press any key to close
```

♥ 2 Likes < Share

Comments (1)

Newest First

Preview POST COMMENT..

 **Newton osita** 2 years ago · 0 Likes

Very wonderful, Coggins, from your bro newton osita

[Newer Post](#)
Introduction to Bell states in Qiskit
with Code (/tutorials/introduction-
to-bell-states)

[Older Post](#)
Truth tables with Grover's Search in
Qiskit (/tutorials/truth-tables-with-
grovers-search-in-qiskit)

[ABOUT US \(/ABOUTUS\)](#)
[CONTACT US \(/CONTACT-US\)](#)

SCARBOROUGH QUANTUM COMPUTING LTD, 26 PRINCE OF WALES TERRACE, SCARBOROUGH, ENGLAND, YO11 2AN,
UNITED KINGDOM

Copyright 2023 Scarborough Quantum Computing Ltd Company Number: 13106408